SingleCell — Reporte paso a paso

Capítulos 1–3, muestras R1 (0N / 0.5N / 5N)

Pipeline SingleCell

2026-06-24

Contents

1 Resumen de la corrida **2**

2 Entorno de trabajo: Docker **3**
- 2.1 Con docker-compose (recomendado): nube o local, a elección . . . 3
- 2.2 Manual, sin docker-compose . . . 3

3 Capítulo 1 — Procesamiento single-cell (R / Seurat) **4**
- 3.1 SECTION 0 — Figura del pipeline . . . 4
- 3.2 SECTION 1 — Carga de datos y QC pre-filtro . . . 4
- 3.3 SECTION 2 — Filtrado de células y doublets . . . 4
- 3.4 SECTION 3 — Merge y preprocesamiento inicial . . . 5
- 3.5 SECTION 4 — Corrección de batch con Harmony . . . 5
- 3.6 SECTION 5 — Optimización de resolución . . . 6
- 3.7 SECTION 6 — Clustering final . . . 7
- 3.8 SECTION 7 — Dotplot de marcadores (guía pre-anotación) . . . 7
- 3.9 SECTION 8 — Anotación de cell type . . . 8
- 3.10 SECTION 9 — Clustree anotado . . . 8
- 3.11 SECTION 10 — Visualización de expresión génica . . . 9
- 3.12 SECTION 11 — Agrupación de cell types [opcional] . . . 9
- 3.13 SECTION 12 — Curación interactiva de cell types [opcional] . . . 10
- 3.14 SECTION 13 — Exportación a h5ad (Python/Scanpy) . . . 11

4 Capítulo 2 — Pseudobulk, DESeq2, GO y redes (R) **12**
- 4.1 SECTION 14 — Subsets por cell type . . . 12
- 4.2 SECTION 15 — Pseudo-réplicas . . . 12
- 4.3 SECTION 16 — Tablas pseudobulk y DESeq2 . . . 12
- 4.4 SECTION 17 — Volcano plots . . . 12
- 4.5 SECTION 18 — Tablas de genes diferenciales . . . 13
- 4.6 SECTION 19 — Enriquecimiento GO . . . 13
- 4.7 SECTION 20 — Heatmap log2FC . . . 13
- 4.8 SECTION 21 — Red de coexpresión (hdWGCNA) . . . 14
- 4.9 SECTION 22 — Exportación y visualización de la red . . . 14

5 Capítulo 3 — Pseudotiempo (Python / scFates) **16**
- 5.1 SECTION 24 — Setup . . . 16
- 5.2 SECTION 25 — Carga del objeto curado . . . 16
- 5.3 SECTION 26 — Selección de cell types . . . 16
- 5.4 SECTION 27 — Inferencia de la trayectoria . . . 17
- 5.5 SECTION 28 — Genes sobre la trayectoria . . . 18

5.6 SECTION 29 — Gene trends: comparación de 4 métodos 19
5.7 Trayectoria alternativa: Stomatal Line → Guard Cell 21

1 Resumen de la corrida

Solo **R1** de las 3 condiciones de nitrógeno (sin réplicas `_2`): `Sample_0N`, `Sample_05N`, `Sample_5N`. Paleta de colores por condición, fijada en `capitulo1_single_cell.R` y usada en todo el pipeline:

0N #66c2a5 0.5N #41ae76 5N #fc8d62

Cada paso siguiente muestra: **qué hace**, el **comando** real usado en el script, y una **imagen pequeña** a modo ilustrativo del resultado.

2 Entorno de trabajo: Docker

Todo lo anterior corre dentro de un contenedor Docker (`matigara/scrnaseq:latest`, R 4.5 + Python 3.12 + Seurat + scanpy + scFates + CellRanger).

2.1 Con docker-compose (recomendado): nube o local, a elección

`docker-compose.yml` declara `image: matigara/scrnaseq:latest` y `build: .` al mismo tiempo — el mismo archivo sirve para las dos vías, sin editar nada:

```
docker compose pull      # usa la imagen de Docker Hub (no construye nada)
docker compose build     # o construye tu propia versión local
                         #   (sobreescribe el mismo tag, solo en este host)

docker compose run --rm r          # abre consola R con lo que haya quedado
docker compose run --rm r bash     # o bash, si se prefiere
```

`docker-compose.yml` monta `~/projects` en `/workspace` (editar esa ruta en el archivo si los datos están en otra carpeta).

2.2 Manual, sin docker-compose

```
docker pull matigara/scrnaseq:latest                  # nube
# o: docker build -t matigara/scrnaseq:latest .       # local

docker run -it -v ~/projects:/workspace matigara/scrnaseq:latest /bin/bash
```

Cualquiera de las dos vías deja una terminal bash dentro del contenedor, con el proyecto disponible en `/workspace`. Desde ahí:

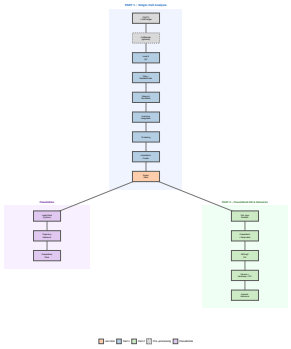
```
R                          # consola interactiva de R
python3                    # consola interactiva de Python
Rscript capitulo1_single_cell.R     # correr un script R completo
python3 capitulo3_pseudotime.py     # correr un script Python completo
```

3 Capítulo 1 — Procesamiento single-cell (R / Seurat)

3.1 SECTION 0 — Figura del pipeline

Diagrama de referencia con el flujo completo de los 3 capítulos (QC → integración → clustering → anotación → pseudobulk/DE → GO → redes → pseudotiempo). Es puramente documental: no alimenta ningún paso posterior, solo orienta al lector antes de empezar.

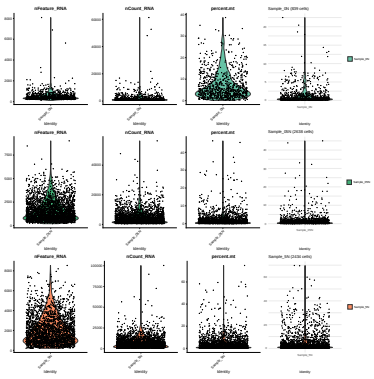
```
plot_pipeline_workflow(file.path(dir_01, "pipeline_workflow.pdf"))
```



3.2 SECTION 1 — Carga de datos y QC pre-filtro

Cada muestra se carga por separado desde su matriz de CellRanger (`filtered_feature_bc_matrix`). Por célula se calculan tres métricas de calidad: `nFeature_RNA` (genes detectados), `nCount_RNA` (UMIs totales) y el % de UMIs mitocondriales/cloroplasto (`mt_pattern="^ATMG"`, `cp_pattern="^ATCG"`) — un % mitocondrial alto suele indicar células moribundas o con membrana rota, y el cloroplasto es contaminación ambiental típica de protoplastos vegetales. Estas distribuciones, graficadas **antes** de filtrar (`qc_prefilter.pdf`), son las que se inspeccionan para decidir los umbrales de la Sección 2.

```
seurat_list_raw <- load_seurat_samples(samples = samples, DATA_DIR = DATA_DIR,
                                       USE_CELLBENDER = FALSE,
                                       mt_pattern = "^ATMG", cp_pattern = "^ATCG")
plot_qc_batch(seurat_list_raw, colors, "qc_prefilter.pdf")
```

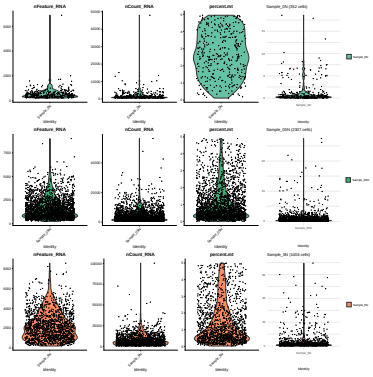


3.3 SECTION 2 — Filtrado de células y doublets

Se eliminan células con menos de `min_features=200` genes detectados (probables droplets vacíos o capturas de baja calidad) o con más de `max_mt=5`% de lecturas mitocondriales (células dañadas). Por separado, DoubletFinder estima, dentro de cada muestra, qué barcodes corresponden probablemente a dos células capturadas juntas (doblete): entrena un clasificador kNN contra dobletes sintéticos generados artificialmente

y descarta los barcodes que caen en esa vecindad. El panel post-filtro (`qc_postfilter.pdf`) muestra las mismas métricas ya limpias, para confirmar visualmente que el corte funcionó.

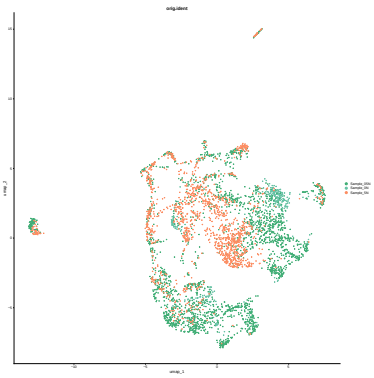
```
seurat_list <- filter_seurat_samples(seurat_list_raw, min_features = 200, max_mt = 5)
plot_qc_batch(seurat_list, colors, "qc_postfilter.pdf")
```



3.4 SECTION 3 — Merge y preprocesamiento inicial

Las muestras filtradas se fusionan en un solo objeto y se preprocesan: log-normalización, selección de los 2000 genes más variables (método VST, que corrige la relación esperada entre media y varianza en vez de quedarse solo con los genes más expresados), escalado (z-score por gen, para que PCA no quede dominado por genes de alta expresión) y PCA con 30 componentes. El UMAP resultante, coloreado por muestra, normalmente separa las células **por muestra** antes que por tipo celular — ese efecto de batch es justamente lo que corrige Harmony en la Sección 4.

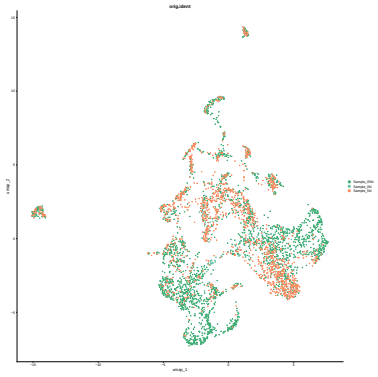
```
pbmc_harmony <- reduce(seurat_list, merge) %>%
  NormalizeData() %>% FindVariableFeatures(nfeatures = 2000) %>%
  ScaleData() %>% RunPCA(npcs = 30) %>% RunUMAP(reduction = "pca", dims = 1:30)
save_pdf(DimPlot(pbmc_harmony, group.by = "orig.ident", cols = colors), "umap_preharmony.pdf")
```



3.5 SECTION 4 — Corrección de batch con Harmony

Harmony corrige el embedding de PCA de forma iterativa: alterna entre un clustering blando (soft k-means) y una corrección lineal por muestra/cluster, hasta que las muestras quedan bien mezcladas dentro de cada grupo sin borrar la variación biológica real entre tipos celulares. A partir de aquí, todas las reducciones (UMAP, vecinos, clustering) usan `"harmony"` en vez de `"pca"`. En el UMAP resultante lo esperable es ver las 3 condiciones (0N/0.5N/5N) mezcladas dentro de cada cluster — contraste directo con el UMAP pre-Harmony de la sección anterior.

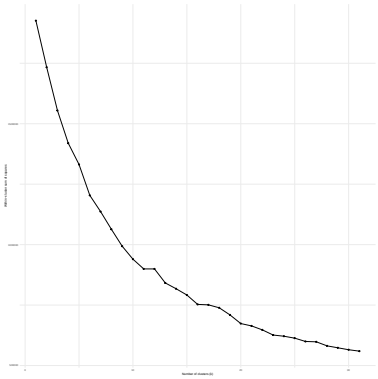
```r
pbmc_harmony <- pbmc_harmony %>%
  RunHarmony("orig.ident", plot_convergence = FALSE) %>%
  RunUMAP(reduction = "harmony", dims = 1:30)
save_pdf(DimPlot(pbmc_harmony, group.by = "orig.ident", cols = colors), "umap_postharmony.pdf")
```

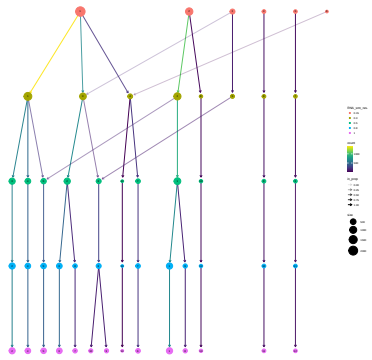


3.6 SECTION 5 — Optimización de resolución

Dos diagnósticos guían la elección de la resolución de clustering. (a) El codo de *within-cluster sum-of-squares* (k-means, k=1–31): el punto de inflexión da una estimación gruesa del número de poblaciones principales. (b) `clustree`: corre Leiden a 5 resoluciones (0.15–1.0) y dibuja cómo se dividen o fusionan los clusters al subir la resolución — una rama que se mantiene estable en varias resoluciones sugiere una subpoblación real, mientras que un cluster que sigue fragmentándose en piezas chicas suele ser sobre-clusterización (ruido, no biología).

```r
wss <- sapply(1:31, function(k) kmeans(pca_data, centers = k, nstart = 4)$tot.withinss)
clu <- pbmc_harmony %>% RunUMAP(...) %>% FindNeighbors(reduction = "harmony", dims = 1:30)
for (res in c(0.15, 0.30, 0.50, 0.8, 1.0))
  clu <- FindClusters(clu, resolution = res, algorithm = 4)
save_pdf(clustree(clu, prefix = "RNA_snn_res."), "clustree.pdf", w = 18, h = 18)
```

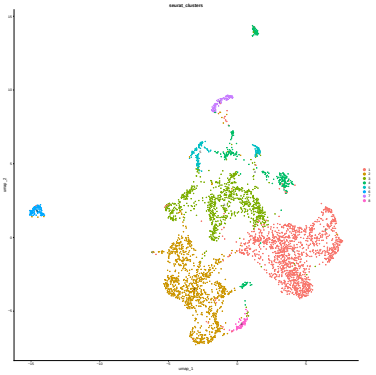




3.7 SECTION 6 — Clustering final

Se aplica la resolución elegida (0.3, algoritmo 4 = Leiden) sobre el grafo de vecinos de Harmony para la asignación final de clusters. Estos números de cluster todavía no son etiquetas biológicas — son solo agrupaciones; las Secciones 7–8 son las que les asignan identidad celular.

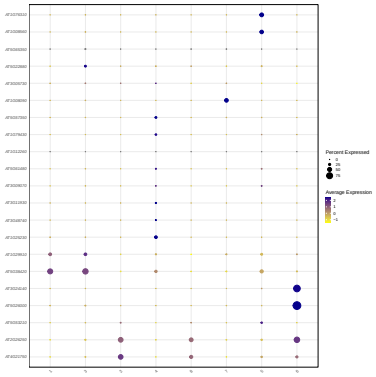
```
pbmc_harmony <- pbmc_harmony %>%
  FindNeighbors(reduction = "harmony", dims = 1:30, k.param = 30) %>%
  FindClusters(resolution = 0.3, algorithm = 4)
save_pdf(DimPlot(pbmc_harmony, group.by = "seurat_clusters", label = TRUE), "umap_seuratclusters.pdf")
```



3.8 SECTION 7 — Dotplot de marcadores (guía pre-anotación)

Muestra la expresión de genes marcadores de bibliografía por cada cluster numerado (tamaño del punto = fracción de células que lo expresan, color = nivel de expresión promedio). Un cluster que expresa fuertemente un marcador conocido (p. ej. un gen específico de Guard Cell) da una pista de su identidad antes de la anotación formal de la Sección 8.

```
plot_marker_dotplot(pbmc_harmony, marker_table, annot_col = "seurat_clusters",
                    outfile = "dotplot_marker_table_preannotation.pdf")
```

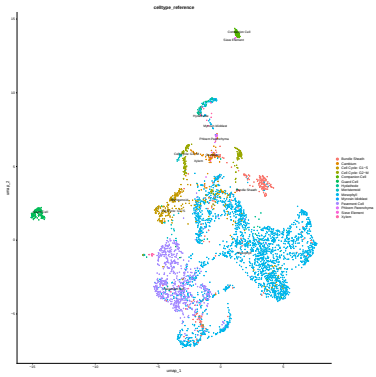


3.9 SECTION 8 — Anotación de cell type

Dos estrategias complementarias. (a) Bibliografía: se corre `FindAllMarkers` para obtener los genes diferenciales de cada cluster y se cruzan contra la tabla de marcadores conocidos, asignando la etiqueta de mejor calce. (b) Referencia: se transfieren etiquetas desde un atlas de hoja de *Arabidopsis* publicado y revisado por pares (GSE273033), usando anchors basados en correlación canónica (`FindTransferAnchors`) y transferencia probabilística (`TransferData`) — en general más objetiva que depender de una lista de marcadores armada a mano. La columna `celltype_reference` (estrategia b) es la que se usa de aquí en adelante.

```
markers      <- find_markers(pbmc_harmony, output_file = "FindAllMarkers.tsv")
pbmc_harmony <- annotate_by_markers(pbmc_harmony, markers, reference_file = biblio_marks_file)

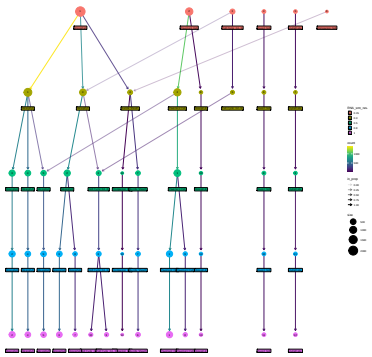
reference_obj <- readRDS("GSE273033_seuratObj_for_publication.rds")
pbmc_harmony  <- annotate_by_reference(pbmc_harmony, reference_obj, reference_col = "annotation")
```



3.10 SECTION 9 — Clustree anotado

Repite el mismo barrido de resoluciones de la Sección 5, pero superponiendo en cada nodo la etiqueta de cell type más frecuente (`Mode`). Sirve para confirmar que la resolución 0.3 no está fusionando dos tipos celulares distintos en un solo cluster, ni partiendo innecesariamente un mismo tipo celular en varios clusters.

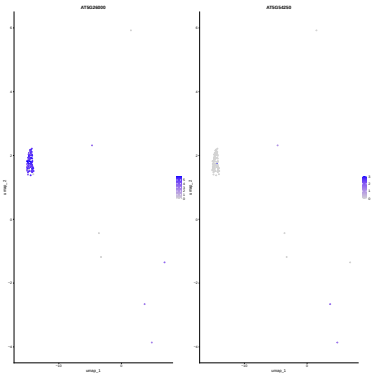
```
save_pdf(clustree(clu, prefix = "RNA_snn_res.",
                  node_label = "celltype_reference", node_label_aggr = "Mode"),
         "clustree_annotated.pdf", w = 18, h = 18)
```



3.11 SECTION 10 — Visualización de expresión génica

Vln/Feature plots para inspeccionar genes puntuales, tanto en todos los cell types como dentro de uno específico de interés (`Guard Cell`). El violín muestra la distribución completa por célula (revela bimodalidad o contaminación dentro de un cluster); el Feature plot muestra dónde se localiza esa expresión sobre el UMAP. Es la forma estándar de validar que un marcador sea realmente específico antes de confiar en él para anotar.

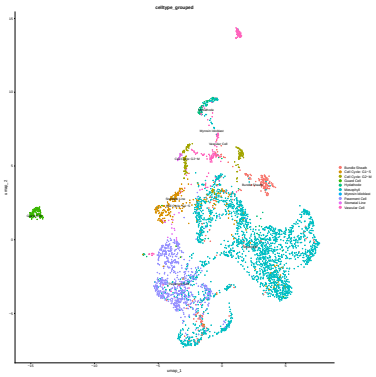
```
sub_obj <- subset(pbmc_harmony, idents = "Guard Cell")
save_pdf(FeaturePlot(sub_obj, features = c("AT5G26000", "AT5G54250")),
         "feature_geneset_celltype.pdf", w = 18, h = 18)
```



3.12 SECTION 11 — Agrupación de cell types [opcional]

Colapsa etiquetas finas en categorías más amplias cuando la distinción no es relevante para el contraste biológico de interés: los distintos subtipos vasculares (Companion Cell, Cambium, Phloem Parenchyma, Xylem, Sieve Element) se agrupan en `"Vascular Cell"`, y `Meristemoid` —el precursor indiferenciado del linaje estomático— se agrupa dentro de `"Stomatal Line"`, anticipando el análisis de trayectoria del capítulo 3.

```
grouping <- c("Companion Cell" = "Vascular Cell", "Cambium" = "Vascular Cell",
              "Phloem Parenchyma" = "Vascular Cell", "Xylem" = "Vascular Cell",
              "Sieve Element" = "Vascular Cell", "Meristemoid" = "Stomatal Line")
pbmc_harmony$celltype_grouped <- recode(pbmc_harmony$celltype_reference, !!!grouping)
```

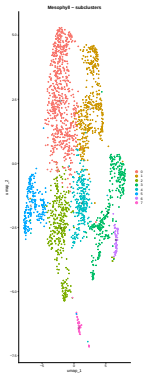


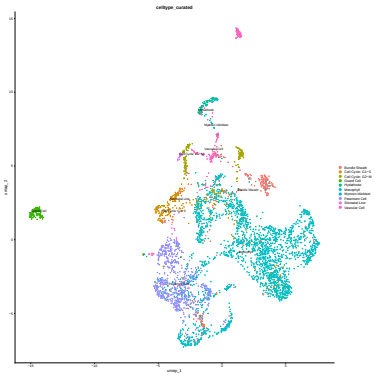
3.13 SECTION 12 — Curación interactiva de cell types [opcional]

Subclusteriza las dos poblaciones que se veían heterogéneas en el UMAP (`Mesophyll`, `Pavement Cell`), para revisar si contienen contaminación de otro tipo celular escondida dentro de la misma etiqueta. Cada subcluster se inspecciona visualmente (figura de abajo) antes de decidir su reasignación final. **En esta corrida la tabla de reasignación es un mapeo identidad** — ningún subcluster mostró señal de contaminación, así que `celltype_curated` termina siendo igual a `celltype_grouped`. Esta sección funciona como un checkpoint de control de calidad, no porque se esperen cambios en cada corrida.

```
mesophyll_umap     <- subcluster_cell_type(pbmc_harmony, "Mesophyll",     annot_col = "celltype_grouped
pavement_cell_umap <- subcluster_cell_type(pbmc_harmony, "Pavement Cell", annot_col = "celltype_grouped

reassign <- list(
  mesophyll_umap     = c("0"="Mesophyll", "1"="Mesophyll", "2"="Mesophyll", others="Mesophyll"),
  pavement_cell_umap = c("0"="Pavement Cell", "1"="Pavement Cell", "2"="Pavement Cell",
                         "3"="Pavement Cell", "4"="Pavement Cell", others="Pavement Cell")
)
pbmc_harmony <- apply_subcluster_reassignment(pbmc_harmony, subcluster_list, reassign,
                                              source_col = "celltype_grouped",
                                              dest_col   = "celltype_curated")
```





3.14 SECTION 13 — Exportación a h5ad (Python/Scanpy)

Convierte el objeto Seurat final a formato AnnData (h5ad), el formato que usa scanpy/scFates en capítulo 3. Se exportan las cuentas normalizadas y las reducciones (PCA, UMAP, Harmony) ya calculadas, para no recalcular nada en Python. Sin figura — esta sección solo escribe el archivo `pbmc_harmony_curated.h5ad`.

```
export_to_scanpy(pbmc_harmony, file.path(dir_objects, "pbmc_harmony_curated.h5ad"))
```

4 Capítulo 2 — Pseudobulk, DESeq2, GO y redes (R)

Capítulo 2 deja de trabajar a nivel de célula individual y agrega las cuentas por pseudo-réplica dentro de cada cell type, para poder usar un test estadístico de bulk RNA-seq (DESeq2) — más robusto que comparar célula a célula cuando lo que importa es la diferencia entre condiciones. Contraste analizado: **0.5N vs 5N**.

4.1 SECTION 14 — Subsets por cell type

Crea un subset de Seurat por cada cell type de `celltype_curated`. El análisis pseudobulk de las secciones siguientes se hace **por separado dentro de cada cell type** — la idea es comparar 0.5N vs 5N específicamente en, por ejemplo, Guard Cell, no mezclar todas las células del experimento en un solo pseudobulk. Sin figura.

```
cell_type_subsets <- create_cell_type_subsets(pbmc_harmony, annot_col = "celltype_curated")
```

4.2 SECTION 15 — Pseudo-réplicas

DESeq2 necesita réplicas biológicas para estimar la dispersión de cada gen, pero el experimento no tiene réplicas reales por condición (una sola muestra de CellRanger por condición). El truco estándar es crear **pseudo-réplicas**: dividir aleatoriamente las células de cada condición (dentro de cada cell type) en 3 grupos, y tratar cada grupo como si fuera una réplica biológica independiente. No es una réplica real — es lo que habilita el test estadístico — por eso el resultado debe leerse con cautela. Sin figura, solo tablas de control (`table(...)`).

```
cell_type_subsets_replicates <- assign_pseudoreplicates_batch(
  cell_type_subsets, pseudobulk_conditions = NULL, n_pseudoreps = 3)
```

4.3 SECTION 16 — Tablas pseudobulk y DESeq2

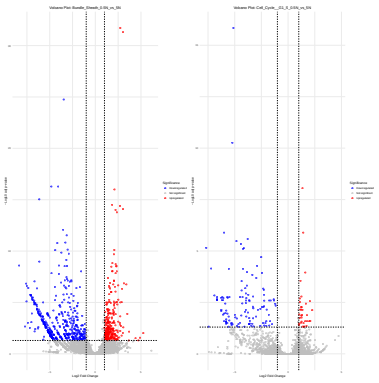
Suma las cuentas crudas de todas las células de cada pseudo-réplica (`AggregateExpression`), generando una matriz genes × pseudo-réplicas por cell type — esto es lo que técnicamente se llama "pseudobulk". Sobre esa matriz, DESeq2 ajusta un modelo binomial negativo por gen y corre un test de Wald para el contraste 0.5N vs 5N. Sin figura — la salida son tablas de resultados (log2FC, p-valor, p-valor ajustado) por cell type.

```
deseq2_results <- run_pseudobulk_deseq2_analysis(
  cell_type_subsets_replicates,
  comparisons = list(list(conds = c("0.5N", "5N"), tag = "0.5N_vs_5N")),
  output_dir  = dir_06)
```

4.4 SECTION 17 — Volcano plots

Un volcano plot por cell type para el contraste 0.5N vs 5N, combinados en un solo PDF. Eje X = log2FC (5N respecto a 0.5N), eje Y = -log10(p-valor ajustado); los puntos coloreados pasan ambos cortes (`padj_cut=0.05`, `lfc_cut=1`) y son los genes que alimentan las tablas y el heatmap de las secciones siguientes.

```
render_volcano_plots(results_dir = file.path(dir_06, "0.5N_vs_5N"),
                     output_dir  = file.path(dir_06, "0.5N_vs_5N", "volcano"),
                     pdf_name = "VolcanoPlots_0.5N_vs_5N.pdf",
                     padj_cut = 0.05, lfc_cut = 1)
```



4.5 SECTION 18 — Tablas de genes diferenciales

Combina los resultados de DESeq2 de todos los cell types en dos tablas únicas: una matriz continua de log2FC (`logfc`, usada en el heatmap de la Sección 20) y una matriz discreta de significancia (-1/0/1 por gen × cell type, usada como universo de genes para GO y para la red de coexpresión). Sin figura.

```
diff_tables <- build_differential_tables(results_dir = file.path(dir_06, "0.5N_vs_5N"),
                                         output_dir  = file.path(dir_06, "0.5N_vs_5N"),
                                         padj_cut = 0.05, lfc_cut = 1)
```

4.6 SECTION 19 — Enriquecimiento GO

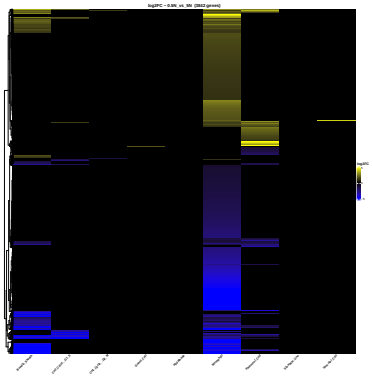
Para cada cell type, toma sus genes significativamente sobre-expresados (de la matriz discreta de la Sección 18) y testea si están enriquecidos en categorías de Gene Ontology (espacio *Biological Process*) frente al universo completo de genes detectados, vía `clusterProfiler::enrichGO` contra `org.At.tair.db`. **En esta corrida no se encontraron términos GO significativos** — usar solo R1 (sin réplicas técnicas) deja pocos genes DE por cell type, lo que reduce mucho el poder estadístico del enriquecimiento. No hay figura para este paso.

```
go_results <- run_go_enrichment_for_contrast(
  results_dir = file.path(dir_06, "0.5N_vs_5N"), output_dir = file.path(dir_07, "0.5N_vs_5N"),
  orgdb = org.At.tair.db, keytype = "TAIR", go_space = "BP", padj_cutoff = 0.05)
```

4.7 SECTION 20 — Heatmap log2FC

Heatmap de log2FC por cell type (filas = genes, columnas = cell types), con los genes ordenados por un dendrograma jerárquico interno (distancia euclidiana + corte dinámico del árbol) que agrupa genes con un patrón de respuesta similar entre cell types. Da una vista panorámica de qué cell types comparten la misma respuesta transcripcional al contraste, y cuáles responden de forma opuesta o exclusiva.

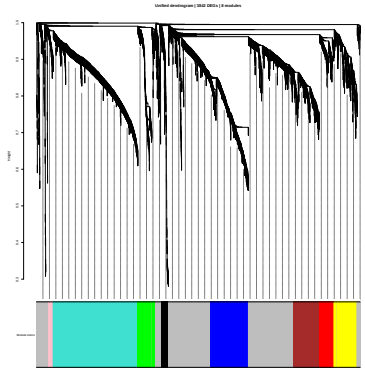
```
build_logfc_heatmap(logfc_table = diff_tables$logfc, contrast_tag = "0.5N_vs_5N",
                    output_dir = file.path(dir_06, "0.5N_vs_5N"), limits = c(-5, 5))
```

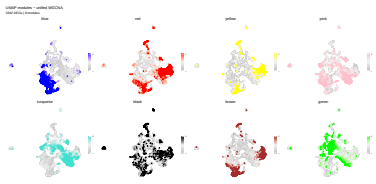


4.8 SECTION 21 — Red de coexpresión (hdWGCNA)

Construye una red de coexpresión (hdWGCNA) sobre el set completo de genes significativos del heatmap anterior. hdWGCNA agrupa células en metacélulas (`n_metacells=25` por cell type × muestra, para reducir el ruido técnico propio de célula única), calcula una matriz de adyacencia ponderada por correlación (TOM) y detecta módulos de genes coexpresados. El dendrograma agrupa genes por módulo; el UMAP de módulos proyecta esos módulos sobre el espacio de expresión para ver en qué cell types se activa cada uno.

```
run_unified_hdwgcna(seurat_obj = pbmc_harmony,
                    de_table_path = file.path(dir_06, "0.5N_vs_5N", "tabla_log2FC_fc1_padj_005.tsv"),
                    output_dir = dir_08, annot_col = "celltype_curated",
                    sample_col = "orig.ident", n_metacells = 25, soft_power = NULL)
```



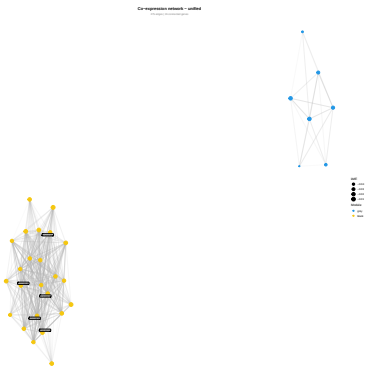


4.9 SECTION 22 — Exportación y visualización de la red

Lee el objeto hdWGCNA unificado, filtra las conexiones por un umbral de TOM (`tom_threshold=0.5` — más alto da una red más simple y solo con conexiones fuertes) y exporta nodos, aristas y el PDF final de la

red, con los `n_hub_label=5` genes más conectados (hubs) de cada módulo etiquetados.

```
plot_hdwgcna_network(hdwgcna_dir = dir_08, output_dir = dir_08,
                     tom_threshold = 0.5, n_hub_label = 5)
```



5 Capítulo 3 — Pseudotiempo (Python / scFates)

Reconstruye un orden continuo de diferenciación (pseudotiempo) entre los cell types seleccionados, ajustando un árbol principal sobre el espacio de difusión de las células y proyectando cada célula sobre ese árbol. A diferencia de capítulo 2 (que compara condiciones), capítulo 3 ignora la condición y se enfoca en la progresión de identidad celular dentro de un linaje.

5.1 SECTION 24 — Setup

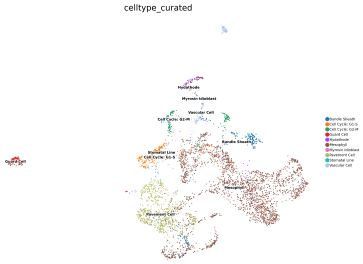
Carga las librerías de Python (scanpy, scFates, Palantir) y las funciones del pipeline de pseudotiempo (`ScRNA_Pseudotime_Functions.py`). Sin figura.

```
exec(open(os.path.join(PIPELINE_DIR, "load_libraries_python.py")).read(), globals())
exec(open(os.path.join(PIPELINE_DIR, "ScRNA_Pseudotime_Functions.py")).read(), globals())
```

5.2 SECTION 25 — Carga del objeto curado

Lee el `.h5ad` exportado en capítulo 1, convierte las coordenadas UMAP/PCA/Harmony al formato que espera scanpy (`adata.obsm`) y lista los cell types disponibles en `celltype_curated` para elegir cuáles entran a la trayectoria en la sección siguiente.

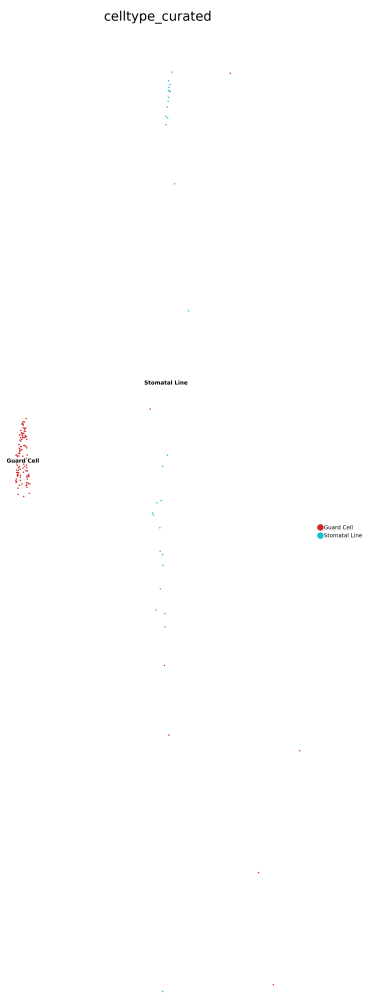
```
adata, N_JOBS = load_curated_object(
    input_h5ad = "/workspace/resultados/objects/pbmc_harmony_curated.h5ad",
    dir_pseudotime = dir_pseudotime, annotation_col = "celltype_curated", n_jobs = 4)
```



5.3 SECTION 26 — Selección de cell types

Subconjunta el objeto a solo los cell types que van a formar la trayectoria: **Pavement Cell + Stomatal Line**. Esta elección encierra una hipótesis biológica de linaje —que ambos tipos derivan de un progenitor común y pueden ordenarse en un continuo de diferenciación— que el árbol de la Sección 27 pone a prueba.

```
TRAJECTORY_CLUSTERS = ["Pavement Cell", "Stomatal Line"]
adata_sub = preview_trajectory_selection(adata, TRAJECTORY_CLUSTERS,
                                         annotation_col = "celltype_curated",
                                         dir_pseudotime = dir_pseudotime)
```



5.4 SECTION 27 — Inferencia de la trayectoria

Construye un árbol principal (PPT/ElPiGraph) sobre el espacio de difusión de las células seleccionadas, y elige automáticamente la célula raíz dentro de `ROOT_CLUSTER` (la de mayor score de indiferenciación). Cuatro parámetros controlan la forma del árbol: `nodes` (más nodos = árbol más fino, sigue más de cerca la nube de puntos), `sigma` (más bajo = se ajusta más estrictamente a las células; más alto = árbol más suave), `lambda` (más alto = árbol más simple, penaliza ramas extra) y `eigs` (dimensiones del mapa de difusión usadas para la geometría de la trayectoria). Las imágenes de abajo ilustran la mecánica del paso con una corrida de ejemplo (`nodes=25, sigma=0.5, lambda=30, eigs=3`); la trayectoria final usada en los resultados de la Sección 29 es `nodes=100, sigma=0.1, lambda=200, eigs=8` — la única combinación, de varias probadas, que produjo un árbol con ramas reales (5 segmentos, 2 bifurcaciones) en vez de un camino lineal simple.

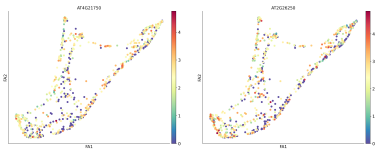
```
ROOT_CLUSTER = "Stomatal Line"
TRAJECTORY_RUNS = [trajectory_run(nodes=25, sigma=0.5, lambda_value=30, eigs=3, seed=3)]
adata_traj, selected_trajectory_dir, trajectory_runs = run_trajectory_runs(
    adata, TRAJECTORY_CLUSTERS, ROOT_CLUSTER, "celltype_curated", dir_pseudotime, TRAJECTORY_RUNS)
```



5.5 SECTION 28 — Genes sobre la trayectoria

Proyecta la expresión de genes de interés sobre el layout de grafo de fuerzas (`draw_graph_fa`) de la trayectoria — un chequeo visual rápido de que un gen candidato efectivamente varía en el sentido esperado del recorrido, antes del ajuste cuantitativo de tendencias de la Sección 29.

```
fig = sc.pl.draw_graph(adata_traj, color = GENES, title = GENES,
                       show = False, return_fig = True)
fig.set_size_inches(18, 18, forward=True)
fig.savefig("step28_genes_FINAL_18x18.png", dpi=200, facecolor="white")
```



5.6 SECTION 29 — Gene trends: comparación de 4 métodos

`scf.tl.test_association()` ajusta, para cada gen, un modelo aditivo generalizado (GAM, `spline_df=5`) de su expresión en función del pseudotiempo, y testea si esa curva se aleja de una línea plana (gen sin relación con la trayectoria). Sobre la trayectoria final de 5 segmentos (`n100_s0.1_l200_e8_seed3`), se compararon 4 criterios para decidir qué genes pasan ese test y se muestran en el heatmap de tendencias:

Método	Alcance	Criterio	Genes
Default estricto	árbol completo	`fdr_cut=0.0001, A_cut=1, st_cut=0.8`	359
Global (estándar)	árbol completo	`A_cut=0.3` + `p_val<0.001` sin FDR	2,486
Por rama	1 rama (`subset_tree`)	`A_cut=0.3` + `p_val<0.001`	1,936
Fork	entre 2 ramas	`test_fork()`	680

```
adata.var["signi"] = adata.var["p_val"] < 0.001      # método global (estándar)
scf.tl.test_association(adata, n_jobs=4, A_cut=0.3)
top_path, custom_path = run_step29_gene_trends(
    adata = adata_traj, run_dir = selected_trajectory_dir,
    name = os.path.basename(selected_trajectory_dir),
    custom_genes = HIGHLIGHT_GENES, top_n = 10, ordering = "max")
```

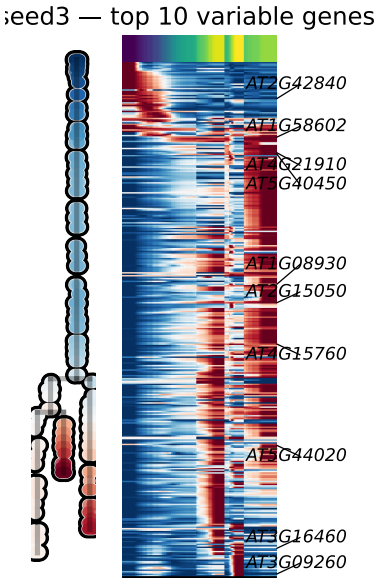


Figure 1: Default estricto (359 genes).

Estándar elegido: Global (`A_cut=0.3` + `p_val<0.001` crudo), integrado por defecto en `run_step29_gene_trends()`, salida en formato 24x18in.

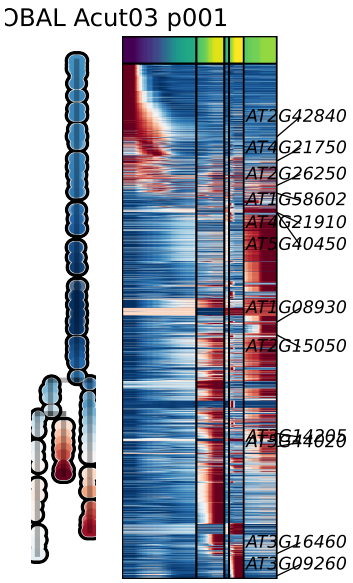


Figure 2: Global — estándar (2,486 genes).

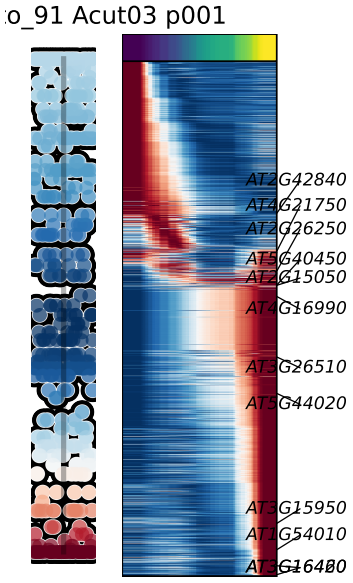


Figure 3: Por rama (1,936 genes).

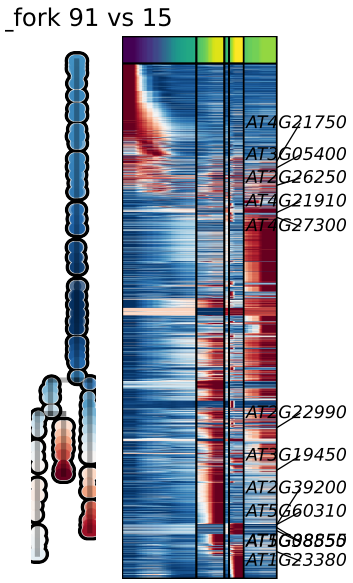


Figure 4: Fork (680 genes).

5.7 Trayectoria alternativa: Stomatal Line → Guard Cell

Misma técnica, seleccionando solo el linaje estomático clásico (Stomatal Line → Guard Cell, sin Pavement Cell). Al ser solo 2 cell types en un linaje directo, sin una tercera población que genere una bifurcación, el árbol resultante es necesariamente un camino lineal (1 segmento, sin bifurcaciones) — 670 genes significativos.

